POWERED BY SCALER School of Technology

INDIA'S BIGGEST MEGA AI HACKATHON

Build at the Bleeding *Edge* of AI

Built on Meta's OpenEnv. The foundation for next-gen RL environments used by leading AI labs.

SPONSORED BY

Meta | Hugging Face | PyTorch

Winners get an interview opportunity at Meta & Hugging Face AI teams

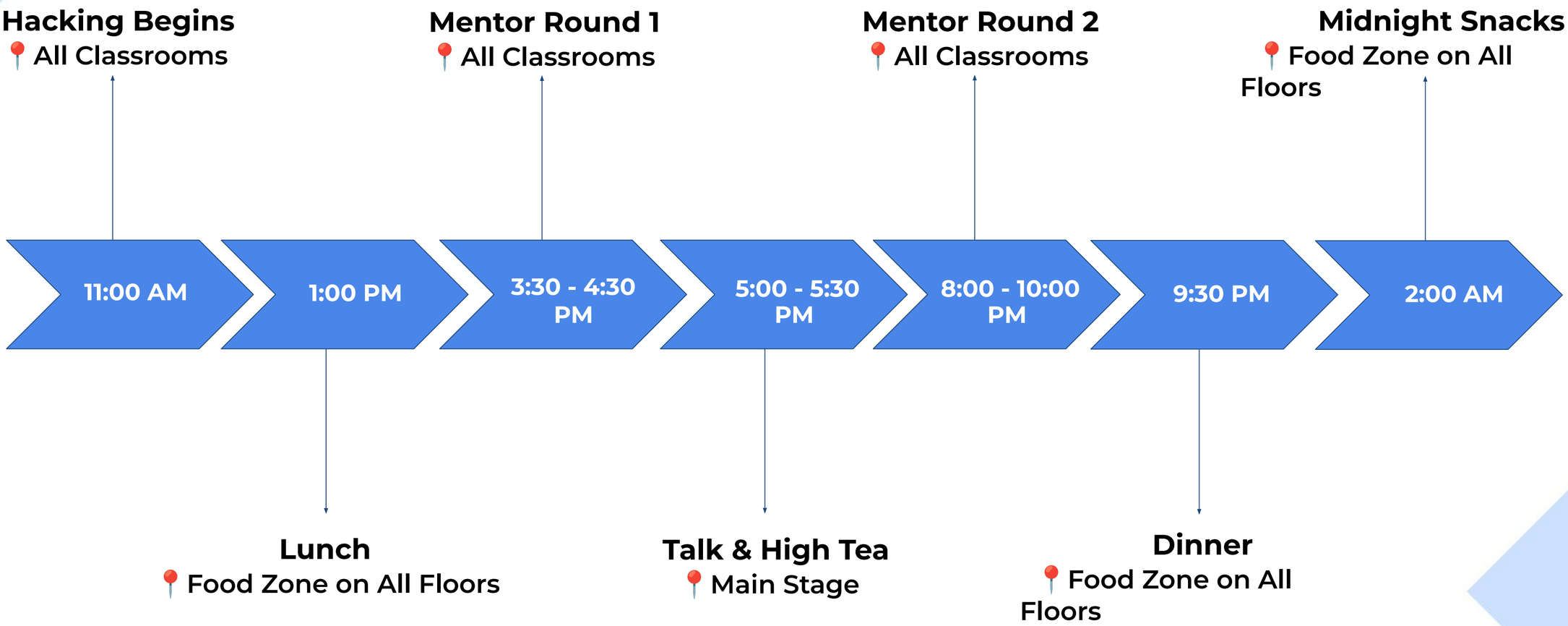
25th March	48 Hours	25th–26th April
ROUND-1 BEGINS	FINALE SPRINT IN BANGALORE	GRAND FINALE

Welcome

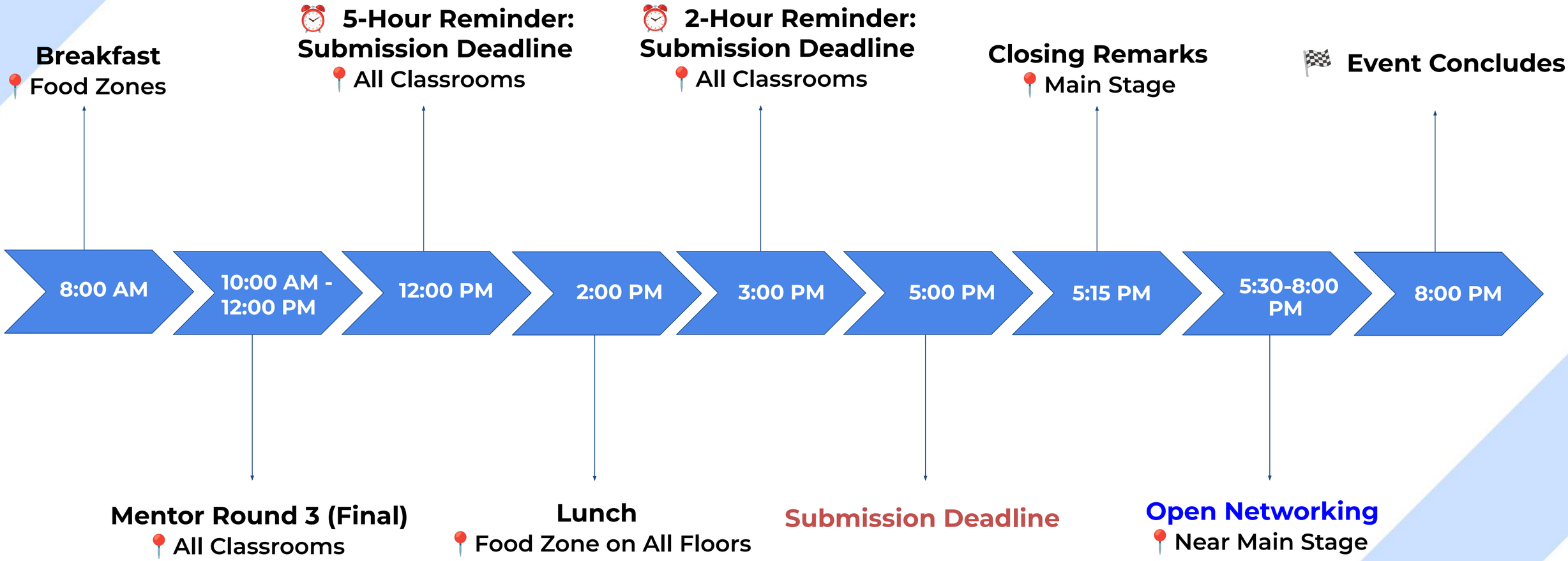
Abhimanyu Saxena

Co-Founder, Scaler

Agenda: Day-1


Hacking Begins
📍 All Classrooms
Mentor Round 1
📍 All Classrooms
Mentor Round 2
📍 All Classrooms
Midnight Snacks
📍 Food Zone on All Floors
11:00 AM
1:00 PM
3:30 - 4:30 PM
5:00 - 5:30 PM
8:00 - 10:00 PM
9:30 PM
2:00 AM
Lunch
📍 Food Zone on All Floors
Talk & High Tea
📍 Main Stage
Dinner
📍 Food Zone on All Floors

Agenda: Day-2


8:00 AM
Breakfast
📍 Food Zones
10:00 AM - 12:00 PM
Mentor Round 3 (Final)
📍 All Classrooms
12:00 PM
⏰ 5-Hour Reminder: Submission Deadline
📍 All Classrooms
2:00 PM
Lunch
📍 Food Zone on All Floors
3:00 PM
⏰ 2-Hour Reminder: Submission Deadline
📍 All Classrooms
5:00 PM
Submission Deadline
5:15 PM
Closing Remarks
📍 Main Stage
5:30-8:00 PM
Open Networking
📍 Near Main Stage
8:00 PM
🏁 Event Concludes

Results Announcement

- Top-100 finalist: Friday, May 1st
- Winners livestream: Friday, May 8th

Cursor AI & HF Credits

Get your credits for Cursor AI and Hugging Face jobs as early as possible

Cursor AI credit: Each participant is eligible to get Cursor credits. Participants can visit Scaler Hackathon dashboard to avail their credits - https://tinyurl.com/sclr-openenv-dashboard

Hugging Face credits → You can avail HF credits by accessing the following link ($30 credit per person):
https://huggingface.co/coupons/claim/hf-openenv-community

The same **links would be shared to you in the #on-campus-discord channels** to access.

Meet your Mentors

Sanyam Bhutani
Partner Engineer, META

Yash Khare
Partner Engineer, META

Nilesh Pandey
Partner Engineer, META

Adithya S Kolavi
Engineer, Hugging Face

Adarsh Shirawalmath
ML Engineer, Hugging Face

Arkadip Maitra
ML Engineer, Red Hat

Aashay Sachdeva
Founding Team, Sarvam

Deepa Dhevannan
Gen AI Solution Architect

Soumik Rakhsit
ML Engineer, Zomato

Meet your Mentors

Ayush Satyam
ML Engineer, Red Hat

Parshant Sharma
ML Engineer, Red Hat

Ben Burtenshaw
Community Education AI,
Hugging Face
(Remotely available)

Alireza Shamsoshoara
PyTorch, Meta
(Remotely available)

Guidelines for Discord

Given that global tech leaders and executives are present in the community, we want to **ensure the highest level of professionalism and decorum is maintained in the community**. We also want to ensure that the incredible talent in this community is represented in the best possible way.

To support this, we have **outlined a set of Do's and Don'ts, and have shared in the community**, make sure to read and adhere to them.

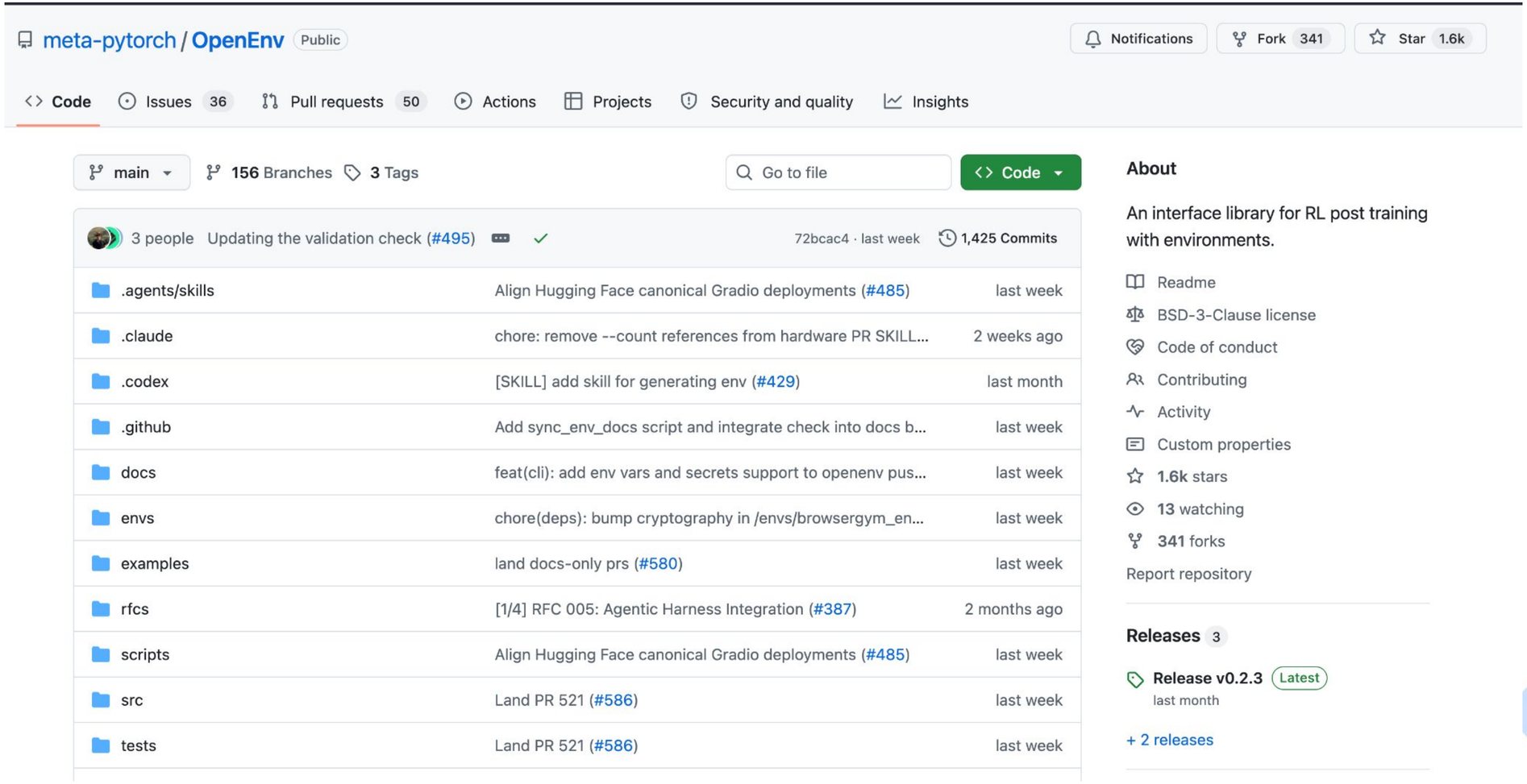
Important: Failure to follow these guidelines will lead to strict action, and may impact your participation in the hackathon.

Welcome

Sanyam Bhutani

Partner Engineer @ Meta

https://tinyurl.com/openenv-scaler



⭐ OpenEnv

Agenda

- PyTorch Foundation Intro
- Hackathon Themes
- Submission and Judging Rules
- RL 101 + OpenEnv Recap
- Best Practises
- Q&A

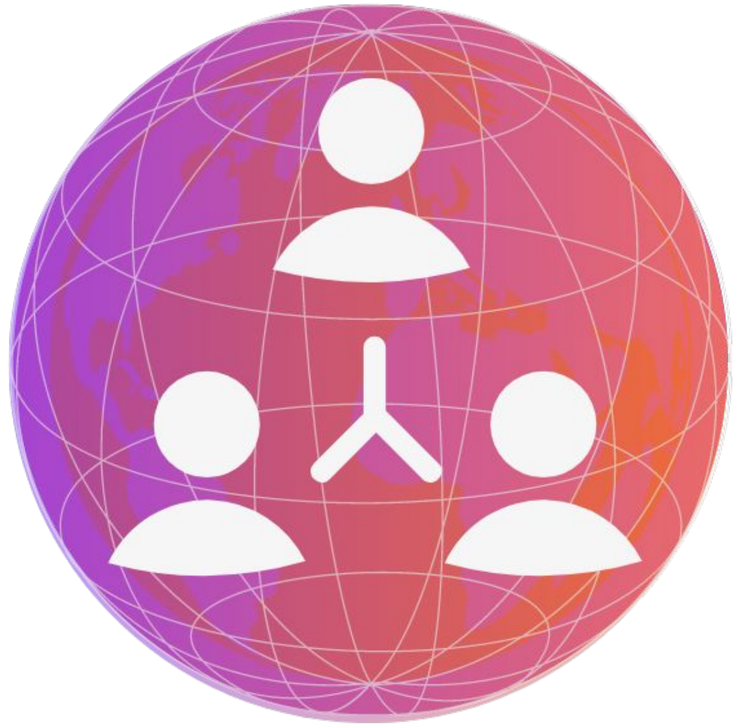
Meta

A bit about PyTorch Foundation

Visit: pytorch.org/join

Mission

The PyTorch Foundation is committed to **democratizing** and **accelerating** the widespread adoption of accessible, high-impact AI technologies by cultivating a robust ecosystem of **open-source, vendor-neutral** projects spanning the entire AI lifecycle —**empowering** researchers, developers, and organizations of all sizes to **innovate and deploy** AI solutions with confidence.



Hosted projects

RAY

vLLM

deepspeed

PyTorch

The Hackathon

Goals:

- Learn RL - now is a great time!
- Hack and create cool environments you can use to add skills to models
- Showcase your work on the Hugging Face Hub
- Have fun!!

https://github.com/meta-pytorch/OpenEnv

Themes

Multi-agent interactions

Market simulations

Compute-allocation negotiations

Collaborative puzzle worlds

..

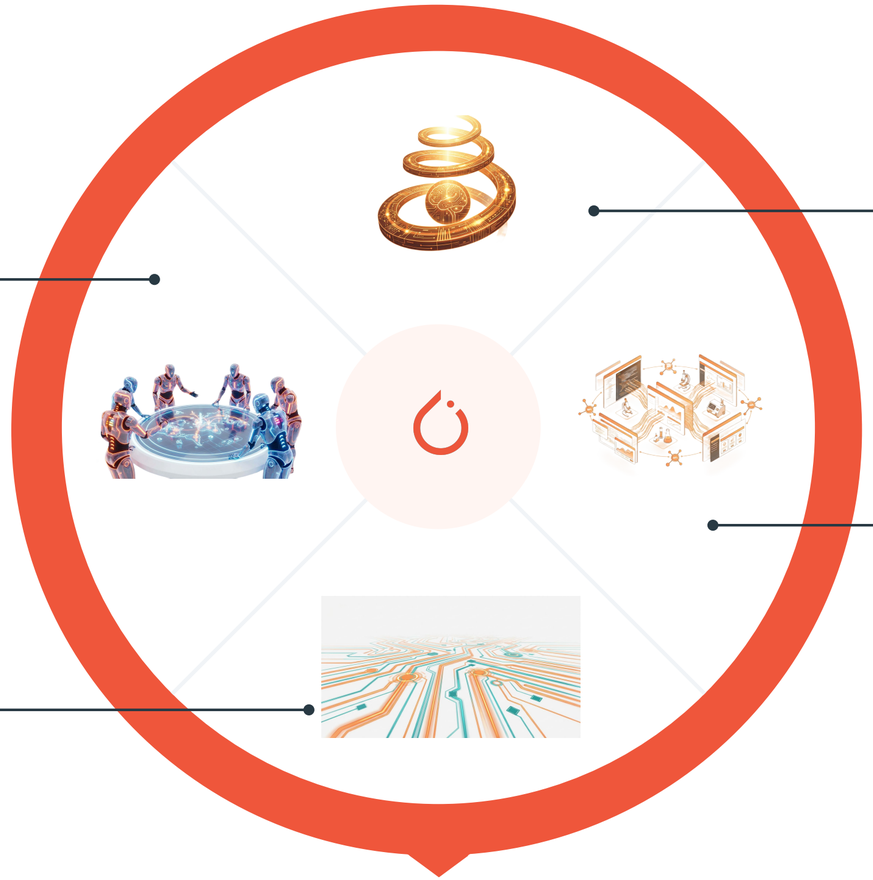
(Super) Long-Horizon Planning

Research-planning simulators

Large-scale codebase refactoring tasks

Strategic resource management worlds

..



Self-Improvement

Self-play negotiation arenas

Auto-generated math/proof tasks

Evolving coding competitions

Adaptive RL curricula

..

World modeling

Dynamic browser/API ecosystems

Enterprise applications

Scientific workflow loops (papers → code → experiments)

..

A Frontier Perspective..

Model Development is Evolving

Model Training (and Inference!) is evolving..

Harness

Infra & Control Tool & API Integration, Safety, Memory, Observability, etc..

Pretraining

Large-scale text/data
Self-supervision loss

SFT

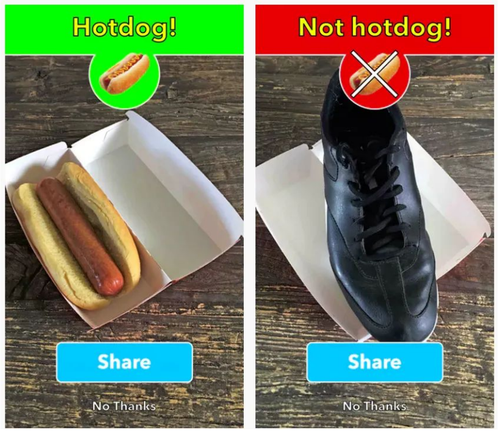
Human-labeled examples
Task-specific tuning

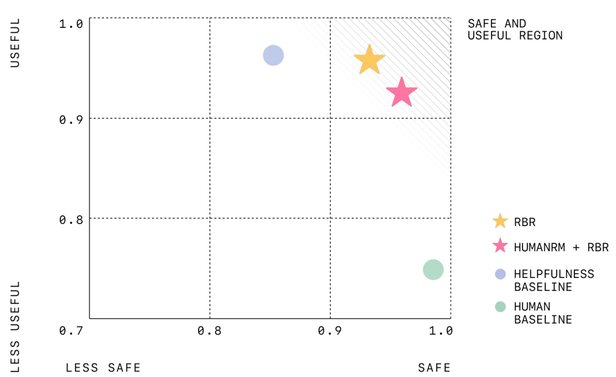
RLHF

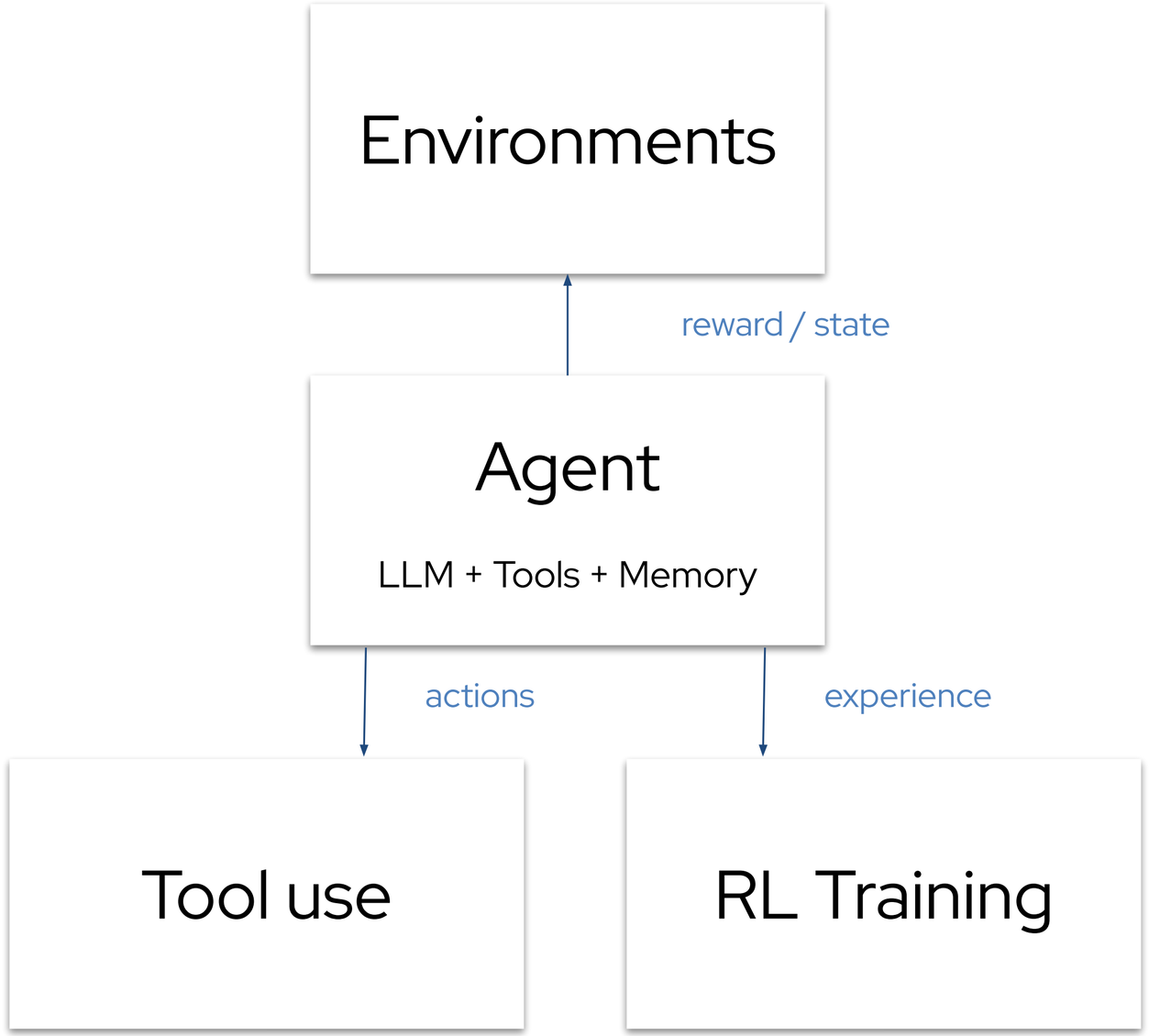
Human preference data
Reward model
Policy optimization

RL + Envs

Agent acts in env
Long-horizon rewards
Self-generated data
Closed-loop learning






Environments
reward / state
Agent
LLM + Tools + Memory
actions
experience
Tool use
RL Training

Now is the time to democratize RL

Hackathon Intro

Guidelines for Problem Statement

It is **NOT** mandatory to choose the same problem statement as Round 1. Only choose the same problem statement if it aligns with the above provided Hackathon themes.

- You can start working on your problem statement once you have finalized it. Post-training can be done onsite on 25th & 26th when you receive compute credits for HuggingFace.
- Before the onsite, we suggest you work on building the environment, agent behaviours, reward model and evaluate if your work aligns with the judging criteria given below.

OpenEnv Hackathon - What Judges Look For

For the list of themes and example problems, refer to the top sections.

NOTE: Please remember **only one submission per team**. If you have multiple ideas, pick the best one and go for it. **Please make sure that the URL link of your environment is submitted as judges will pull the environment from the URL to evaluate it**. Changes or commits after the submission deadline will not be considered.

TL;DR
Build an environment that an LLM could actually be trained on to get measurably better at something interesting. Then show that training. Then tell the story.

A messy but ambitious environment with real training evidence beats a polished but boring one.
Pick a problem that excites you (that energy comes through in the pitch).

Judging Criteria

Criterion: Environment Innovation
Weight: 40%
What it means:
Is the environment novel, creative, or genuinely challenging?
Does it meaningfully test agent behavior **in** a way that hasn't been done before?

Criterion: Showing Improvement in Rewards
Weight: 20%
What it means:
Is there observable evidence of training progress?
Reward curves, before/after behavior, comparison against a baseline -- anything that proves the agent learned something.

Criterion: Storytelling & Presentation
Weight: 30%
What it means:
Can you clearly explain the problem, the environment, and what the agent learned?
Is the demo engaging and easy to follow **for** a non-technical audience?

Criterion: Reward & Training Pipeline
Weight: 10%
What it means:
Is the reward logic coherent? Does the pipeline produce meaningful improvement **in** the trained agent's behavior?

Minimum Submission Requirements

NOTE: These are **non-negotiable**. Submissions missing any of these are at a serious disadvantage.

- Use OpenEnv (latest release). Build on top of the framework; don’t reinvent the wheel.
- A working training script using Unsloth or Hugging Face TRL, ideally as a Colab notebook so judges can re-run it.
- Evidence that you actually trained; at minimum, loss and reward plots from a real run.
- A short writeup: a mini-blog on Hugging Face or a < 2 minute video on YouTube explaining what your environment does and what you trained, or a short slide deck of presentation. Please make sure that all materials are linked from your README file so that judges can access them easily.
- Push your environment to a Hugging Face Space so it’s discoverable and runnable.
- A README that motivates the problem, explains how the env works, and shows results.
  - README should have a link to the environment in the Hugging Face Space. It should also have all additional references to other materials (e.g. videos, blog posts, slides, presentations, etc.) that you want to include.
- Please do not include big video files in your Env submission on HF Hub as we would like to have a small size for each env (Please use url as reference link to additional materials).

What Makes a Submission Stand Out

Pick an ambitious, original problem

The themes (problems) are deliberately open. Use them as launching pads, not boxes. Judges have seen a lot of chess, snake, tic-tac-toe, and grid-world clones. To score well on innovation,
you need a genuinely fresh angle. Some questions to ask yourself:

- Does this environment exist to teach an LLM something it currently can’t do well?
- Is the domain underexplored in RL/LLM training?
- Could a researcher write a paper about training on this?

Design a reward signal that actually teaches

A great environment has a reward function that:

- Provides a rich, informative signal (not just 0/1 at the end)
- Captures something hard to measure in a clever way
- Uses OpenEnv’s Rubric system thoughtfully (composable rubrics > monolithic scoring)
- Is hard to game; an agent that exploits the reward without solving the task should not get high scores

What Makes a Submission Stand Out

Show real training, end to end

The bar isn't "training script exists." The bar is "training script runs against the environment, the agent learns, and you can show it." Concretely:

- Your training loop should connect to your environment (not a static dataset)
- Train long enough that the curves mean something
- Compare a trained agent vs. a random/untrained baseline; quantitative and/or qualitative
- Include the plots and numbers in your README and writeup

Make your plots readable

Reviewers spend seconds, not minutes, on each plot. Help them out:

- Label both axes (e.g. "training step" / "episode" on x, "reward" / "loss" on y) and include units where they apply
- Save plots as .png or .jpg and commit them to the repo (don't leave them only in a Colab cell or a deleted Wandb run) (if you ran via WANBD, please include the link to that specific run of your plots)
- Embed the key plots in your README with a one-line caption explaining what each one shows If you have multiple runs (baseline vs. trained, ablations, etc.), put them on the same axes so the comparison is obvious

What Makes a Submission Stand Out

Tell a story, not an API doc

Your README, blog, and pitch should answer:

1. Problem) what capability gap or interesting domain are you targeting?
2. Environment) what does the agent see, do, and get rewarded for?
3. Results) what changed after training? Show it.
4. Why does it matter) who would care, and why?

A reviewer should be able to read your README in 3~5 minutes and want to try your environment.

NOTE: If you have a video, HF post, or anything else interesting, please make sure that it's linked from your README.

What Makes a Submission Stand Out

Engineer it cleanly (table stakes)

Engineering quality matters less than ambition, but sloppy work hurts. Make sure you:

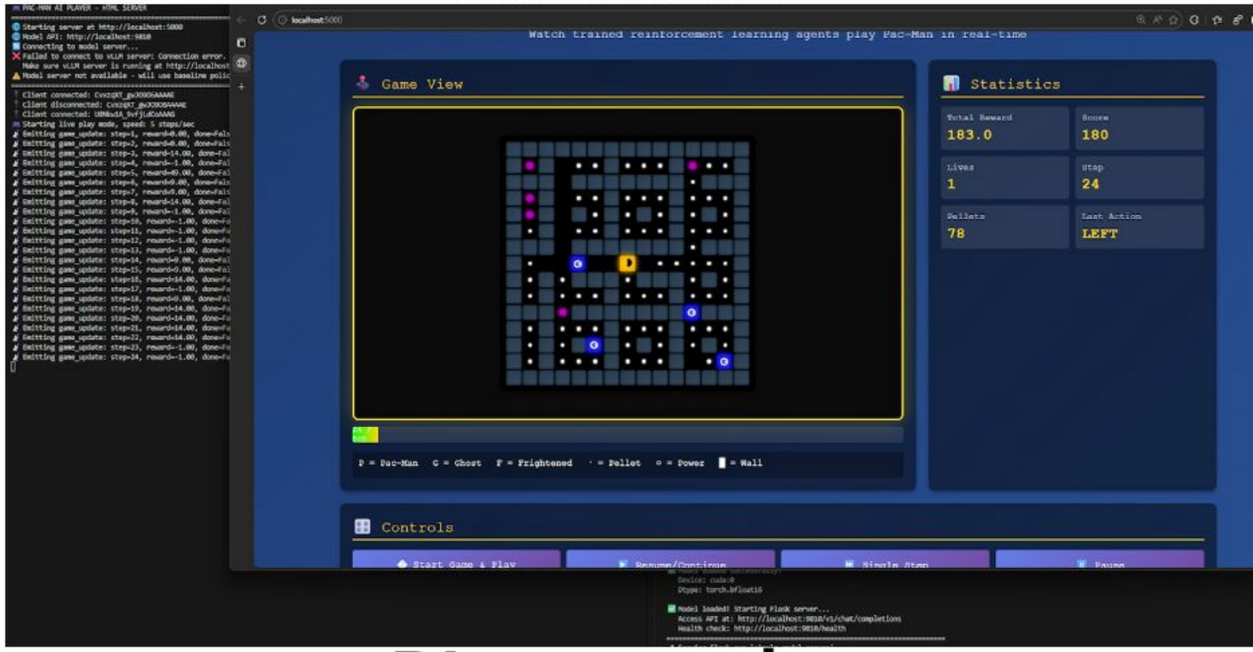
- Use OpenEnv's Environment / MCPEnvironment base classes properly
- Respect the client / server separation (clients should never import server internals)
- Follow the standard Gym-style API (reset, step, state)
- Have a valid openenv.yaml manifest
- Don't use reserved tool names (reset, step, state, close) for MCP tools

Final Note

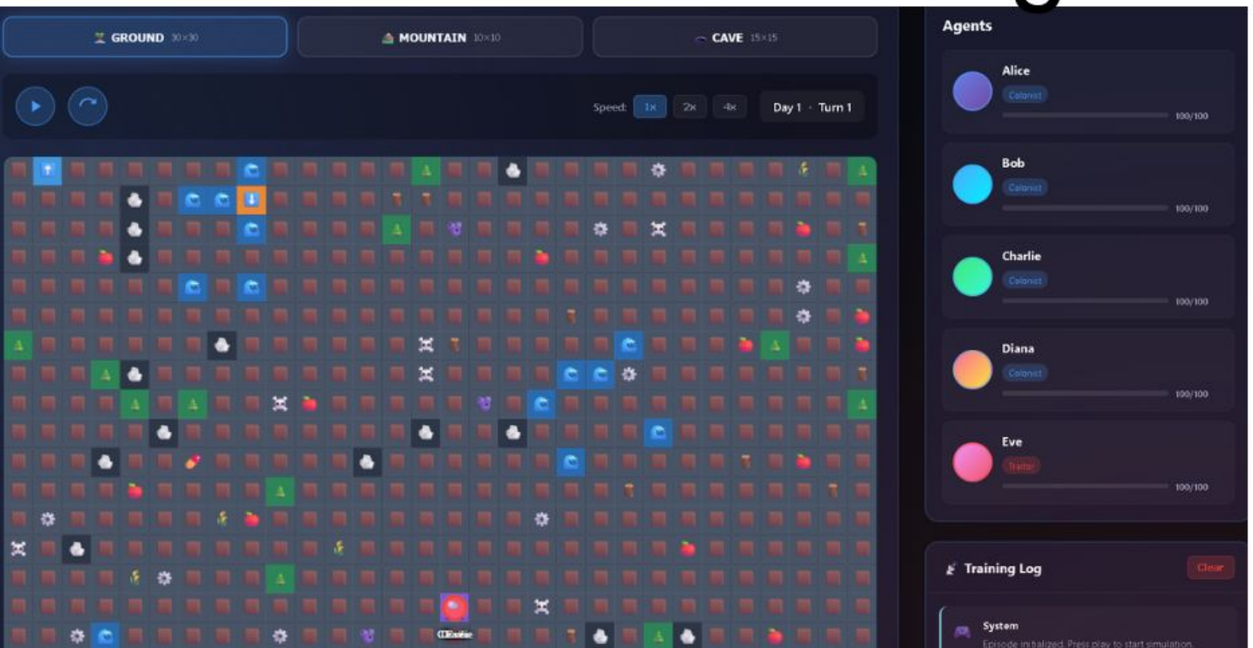
Judges are looking for environments that push the frontier of what we can train LLMs to do. Be ambitious. Pick a problem you find genuinely interesting; that almost always produces better work than chasing what you think judges want. Good luck.

Playing Mario with RL!

Pacman



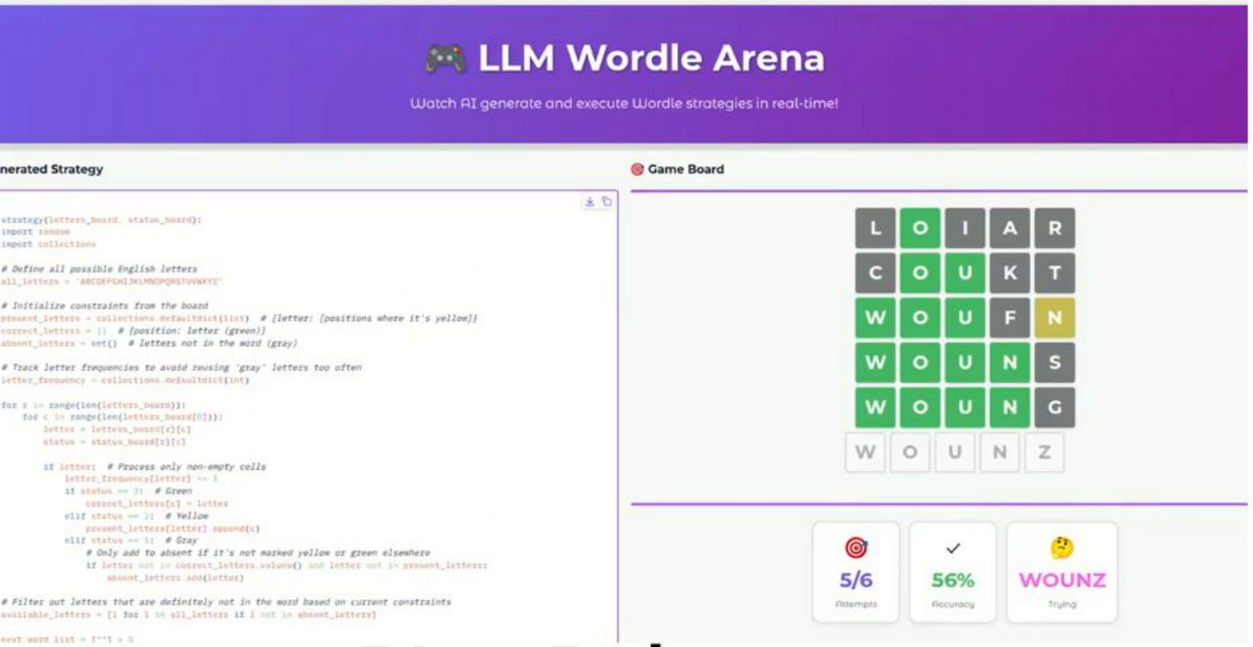
Survivial Island - Multi Agents



RL on code

Language	Baseline	RL
Julia	37%	47% (+10%)
R	23%	31% (+8%)
Ruby	41%	48% (+7%)
Zig	3%	27% (+24%)

Auto RL + Wordle



Chess Strategy Generator

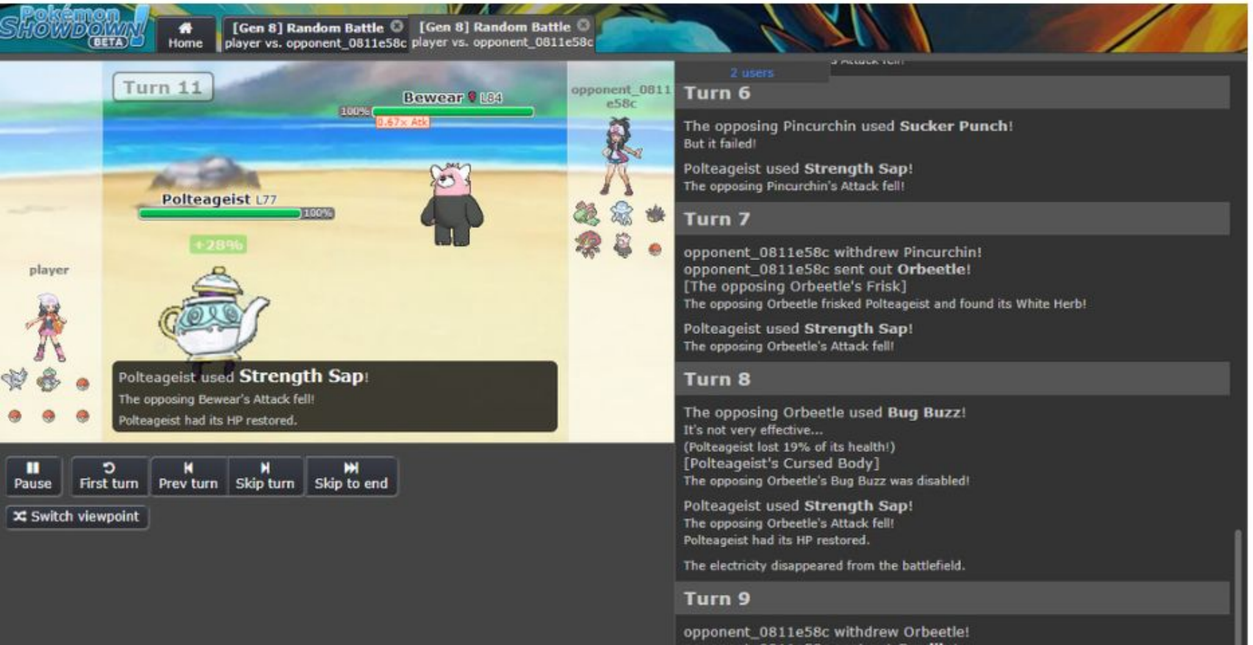
```
   \\   /|    Num examples = 1,000 | Num Epochs = 1 | Total steps = 200
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 1
\         /    Data Parallel GPUs = 1 | Total batch size (2 x 1 x 1) = 2
 "-____-"     Trainable parameters = 3,981,312 of 20,918,738,496 (0.02% trained)
```

```
============================================================
WHITE STRATEGY:
def strategy(board_state, legal_actions):
    # piece value mapping (lowercase for black)
    values = {'P':1,'N':3,'B':3,'R':5,'Q':9,'K':float('inf'),
              'p':1,'n':3,'b':3,'r':5,'q':9,'k':float('inf')}

    # parse FEN to 8x8 board
    board = []
    rows = board_state.split()[0].spl
============================================================

[White] Steps: 0, Outcome: invalid_format, Illegal W/B: 1/0

[White] Steps: 0, Outcome: invalid_format, Illegal W/B: 1/0
```

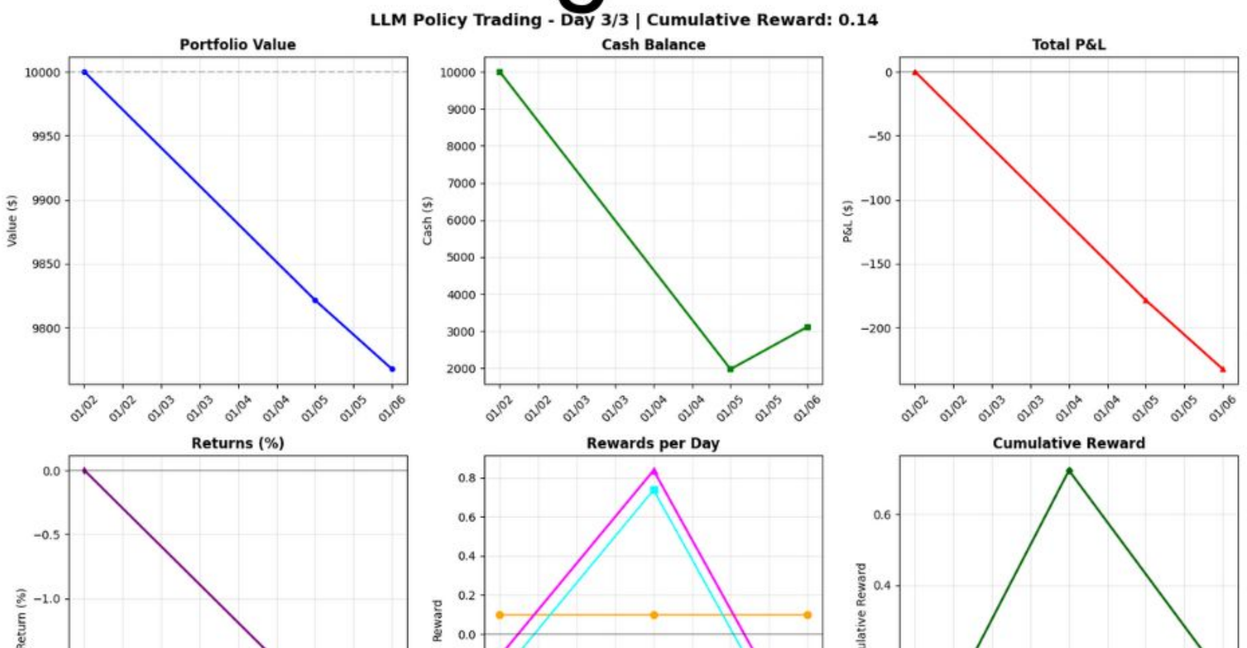
[200/200 51:56, Epoch 0/1]

Step	Training Loss	reward	reward_std	completions / mean_length	completions / min_length	completions / max_length	completions / clipped_ratio	completions / mean_terminated_length	completio min_terminated_len
1	0.000000	-23.500000	14.849242	357.500000	302.000000	413.000000	0.000000	357.500000	302.000
2	0.000000	-0.500000	17.677670	313.000000	185.000000	441.000000	0.000000	313.000000	185.000
3	0.000000	19.500000	45.961941	378.000000	192.000000	564.000000	0.000000	378.000000	192.000

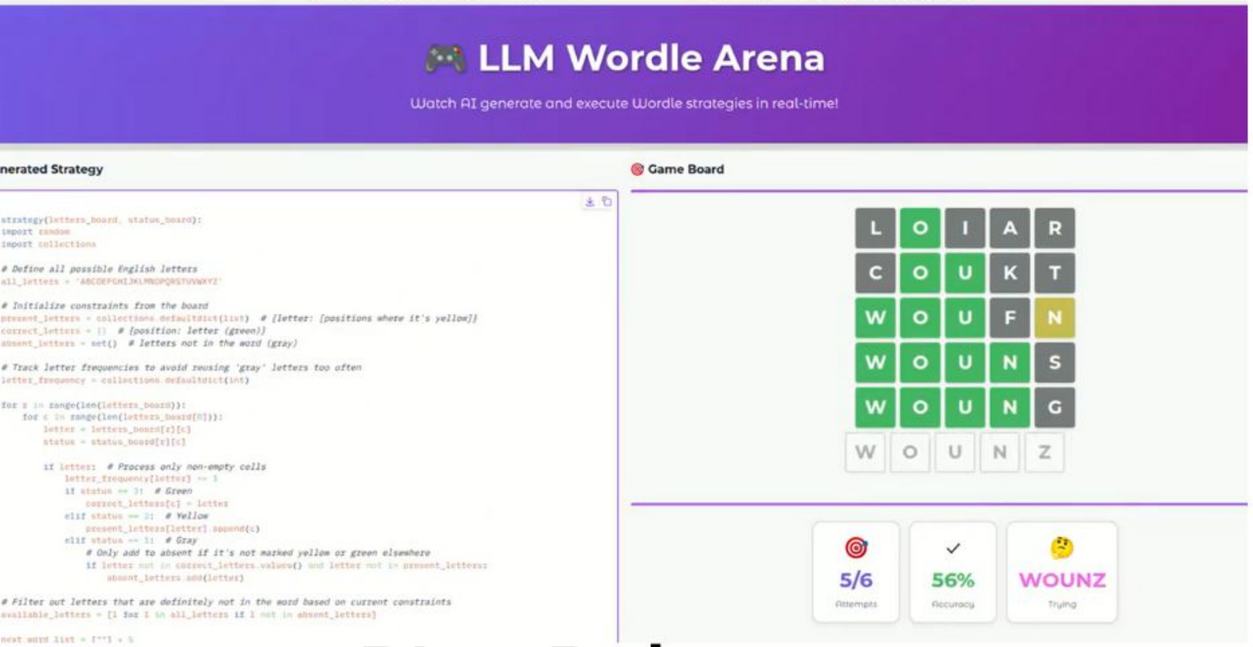
RL + Pokemon



Trading with RL



Auto RL + Wordle



Chess Strategy Generator

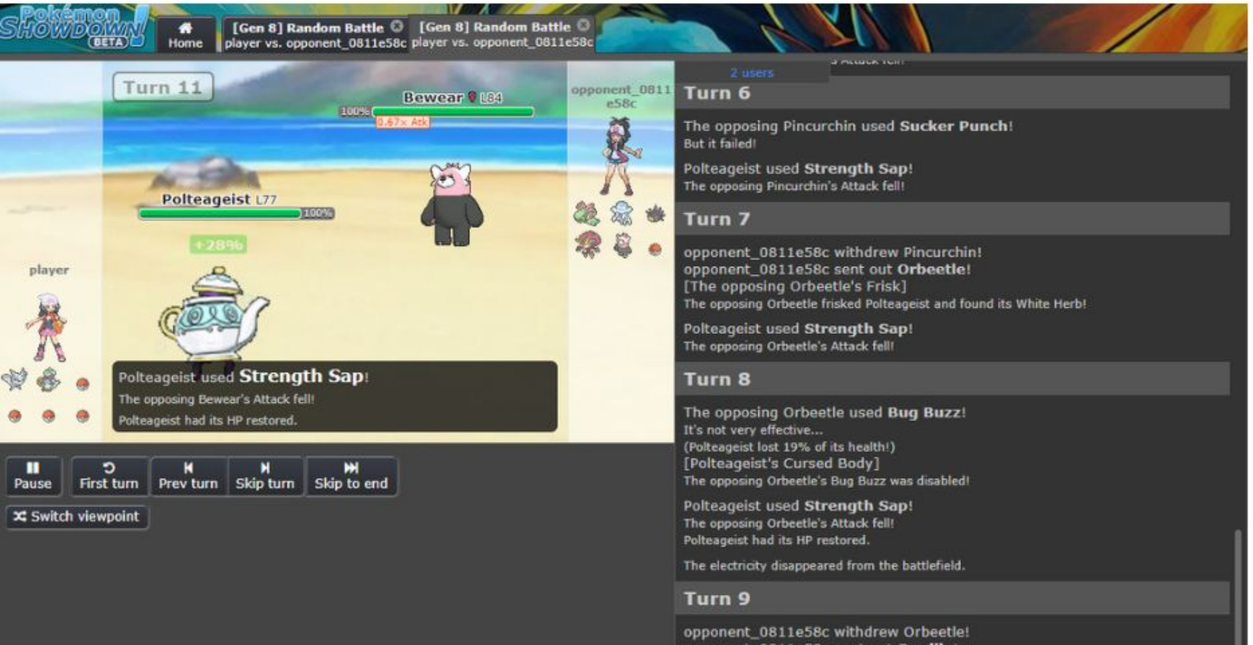
```
   \\   /|    Num examples = 1,000 | Num Epochs = 1 | Total steps = 200
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 1
\         /    Data Parallel GPUs = 1 | Total batch size (2 x 1 x 1) = 2
 "-____-"     Trainable parameters = 3,981,312 of 20,918,738,496 (0.02% trained)
```

```
============================================================
WHITE STRATEGY:
def strategy(board_state, legal_actions):
    # piece value mapping (lowercase for black)
    values = {'P':1,'N':3,'B':3,'R':5,'Q':9,'K':float('inf'),
              'p':1,'n':3,'b':3,'r':5,'q':9,'k':float('inf')}

    # parse FEN to 8x8 board
    board = []
    rows = board_state.split()[0].spl
============================================================

[White] Steps: 0, Outcome: invalid_format, Illegal W/B: 1/0

[White] Steps: 0, Outcome: invalid_format, Illegal W/B: 1/0
```

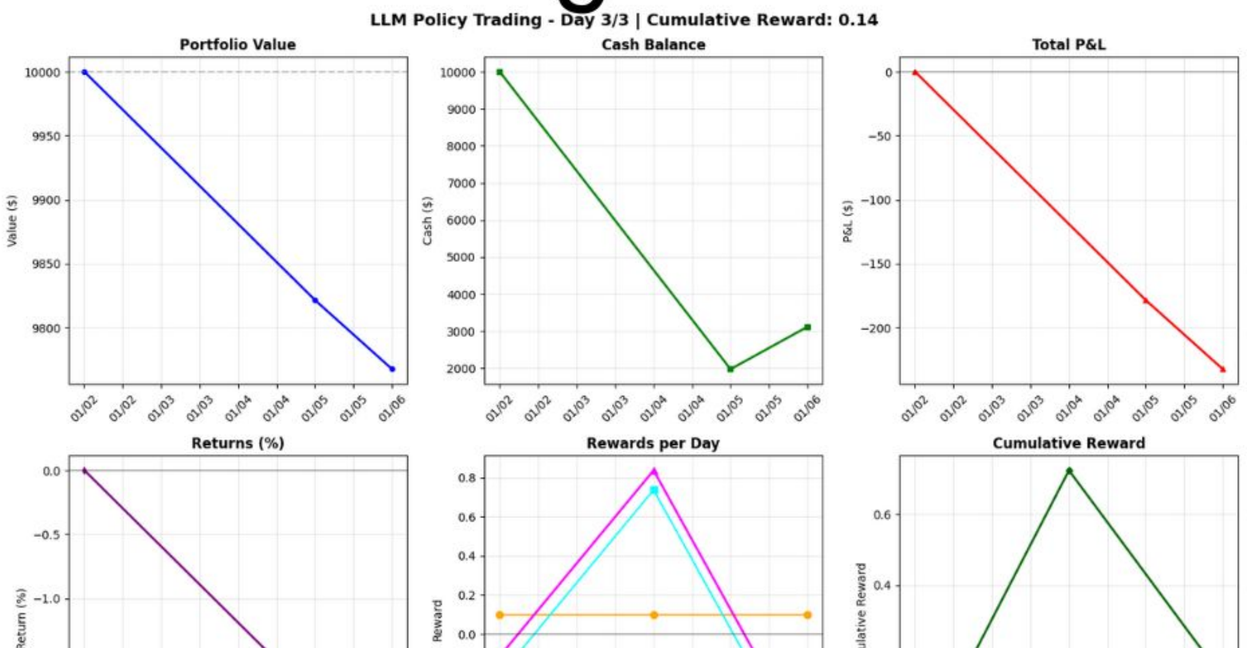
[200/200 51:56, Epoch 0/1]

Step	Training Loss	reward	reward_std	completions / mean_length	completions / min_length	completions / max_length	completions / clipped_ratio	completions / mean_terminated_length	completio min_terminated_len
1	0.000000	-23.500000	14.849242	357.500000	302.000000	413.000000	0.000000	357.500000	302.000
2	0.000000	-0.500000	17.677670	313.000000	185.000000	441.000000	0.000000	313.000000	185.000
3	0.000000	19.500000	45.961941	378.000000	192.000000	564.000000	0.000000	378.000000	192.000

RL + Pokemon



Trading with RL



RL Recap


AGENT
(The Learning Brain)
Observes the world
Chooses actions
Learns from feedback
Action
(e.g., "move left")
Observation
+ Reward
ENVIRONMENT
(The World/Game)
Executes action
Returns new observation
Gives reward (+1, 0, -1)

```
# The RL Loop – Teaching a Dog to Sit 🐕

# Start a new episode
observation = environment.reset()
# "Okay, dog, we're starting a new training session"

while not done:
    # Agent looks at the world
    observation = environment.observe()
    # "What do I see? A human saying 'sit'"

    # Agent decides what to do
    action = agent.choose(observation)
    # "I'm going to... spin in a circle!" 🌀

    # Environment responds
    result = environment.step(action)
    reward = result.reward
    # "No treat. Try again." ❌

    # Agent learns
    agent.learn(reward)
```

The Four Key Concepts

Every RL system uses these

1 `reset()`

Start a New Episode

Begin a fresh training session

2 `observation`

What the Agent Sees

The current state of the world

3 `action`

What the Agent Does

Sit, spin, bark, move left, etc.

4 `step(action)`

Execute the Action

Returns three things:

- New observation
- Reward (+1, 0, -1)
- done flag (episode over?)

This Pattern is Universal

Observe → **Act** → **Reward** → **Learn** → **Repeat**

Building Your Environment

🛠️ Build Your Own Environment

5 Simple Steps

1 📝 **Define Types**
`models.py`
Action, Observation, State dataclasses

2 🏗️ **Implement Environment**
`server/environment.py`
reset(), step(), state() methods

3 📱 **Create Client**
`client.py`
HTTPEnvClient subclass

4 🖥️ **Create Server**
`server/app.py`
app = create_fastapi_app(env)

5 🐋 **Dockerize**
`Dockerfile`
Standard container setup

✨ Or use the CLI! ✨

```
$ openenv init my_env
```

Scaffolding ready in seconds! 🚀

The Universal Interface

Every OpenEnv environment implements these 3 methods

```python
# The contract: All environments must implement this

class Environment:

    def reset(self) -> Observation:
        """Start a new episode"""

    def step(self, action: Action) -> Observation:
        """Execute action, return observation"""

    def state(self) -> State:
        """Get episode metadata"""
```

Type-Safe by Design

Define your data structures with Python dataclasses

🎯 Action

```python
@dataclass
class Action:
    # What the agent does
    move: str
    column: int
    direction: str
```

What the agent can do - move, jump, click, type, etc.

👁️ Observation

```python
@dataclass
class Observation:
    # What the agent sees
    state: List[int]
    done: bool
    reward: float
```

What the agent sees - board state, pixels, text, etc.

📊 State

```python
@dataclass
class State:
    # Episode metadata
    episode_id: str
    step_count: int
    timestamp: float
```

Episode metadata - ID, step count, timing info.

✨ **Your IDE autocompletes • Type checker catches bugs • No mystery numpy arrays** ✨

Same Code, Every Environment

This pattern works for Chess, Atari, Trading, Android - everything

```python
# Works for ANY OpenEnv environment

# Connect to environment (runs in Docker container)
env = SomeEnv.from_docker_image("some-env:latest")

# Start new episode
result = env.reset()

# Take action
action = SomeAction(...)
result = env.step(action)

# Get episode metadata
state = env.state()

# Clean up
env.close()  # Container stops automatically
```

🚀 **Write once, train anywhere • No HTTP code • No JSON parsing • Just clean Python** 🚀

Models → Docker → FastAPI

```python
# models.py base
class Action:
```

extends

```python
@dataclass
class Connect4Action(Action):
  column: int
```

```python
# models.py base
class Observation:
```

extends

```python
@dataclass
class Connect4Observation(Observation):
  board: List[List[int]]
  legal_actions: List[int]
  done: bool
```

```python
# models.py base
class State:
```

extends

```python
@dataclass
class Connect4State(State):
  episode_id: str
  board: List[List[int]]
  next_player: int
```

packages into

Dockerfile

```
FROM openenv-base:latest

# Install dependencies
RUN pip install numpy

# Copy environment code
COPY src/envs/connect4_env/ ...

# Run FastAPI server
CMD ["uvicorn", "app:app"]
```

FastAPI Server

```python
# app.py
from core.env_server import
create_app

env = Connect4Environment()

app = create_fastapi_app(
  env,
  Connect4Action,
  Connect4Observation
)
```

HTTP API

POST /reset

POST /step

GET /state

GET /health

🎮 Connect4 Logic Flow

How It Works - Conceptual

reset()

- 📋 Create empty 6×7 board
- 👤 Set player to 1
- ✅ All columns are legal
- 🎯 Return observation

→

step()

- ⬇️ Drop piece in column
- 🔍 **Check if 4 in a row**
- 🏆 Win? Reward +1
- 🔄 Switch players

→

_check_win()

- → Horizontal
- ↓ Vertical
- ↘ Diagonal down
- ↗ Diagonal up
- 🎉 **4+ in a row = WIN!**

Connect4 Code

environment.py

reset()

```
def reset(self):
  # Empty 6x7 board
  self.board = zeros((6, 7))
  self.next_player = 1

  return Connect4Observation(
    board=self.board,
    legal_actions=
[0,1,2,3,4,5,6],
    done=False
  )
```

step()

```
def step(self, action):
  col = action.column

  # Drop piece down

for row in range(5, -1, -1):

if self.board[row, col] == 0:

self.board[row, col] = self.next_
      break

  # Check win

won = self._check_win(row, col)

  # Switch players
  self.next_player *= -1
```

_check_win()

```
def _check_win(self, row, col):
  # Check 4 directions
  dirs = [
    (1,0), # →
    (0,1), # ↓
    (1,1), # ↘
    (1,-1) # ↗
  ]

  for dr, dc in dirs:

if count_consecutive() >= 4:
      return True  # WIN!

  return False
```

🔥 Kernel Sandbox Logic Flow

How It Works - Conceptual

`reset()`

- 🎮 Detect **GPU model**
- 📊 Run **cuBLAS** baseline
- ⚡ Record baseline **TFLOPS**
- 🎯 Return observation

→

`step()`

- ⬇️ Receive **kernel code**
- 🔧 **Compile** with **flags**
- 🚴 Run benchmark
- 🔁 Call **_benchmark()**

→

`_benchmark()`

- → Measure **TFLOPS**
- ↓ Measure **bandwidth**
- ↘ Check **occupancy**
- ↗ Check **registers**
- 🏆 **speedup = kernel / baseline**

🔥 **Return: tflops, occupancy, speedup**

Conceptual Example: Kernel Sandbox

How the pattern looks for GPU kernel optimization

📝 models.py

```python
@dataclass
class KernelAction(Action):
    kernel_code: str
    block_size: Tuple[int,int,int]
    compiler_flags: List[str]

@dataclass
class KernelObservation(Observation):
    compiled: bool
    tflops: float
    occupancy: float
    speedup: float
# + KernelState...
```

🏗️ environment.py

```python
class KernelSandbox(Environment):

    def reset(self):
        self.baseline = self._run_baseline()
        return self._observe()

    def step(self, action):
        # Compile, benchmark, profile
        # Return TFLOPS, occupancy
```

🖥️ server/app.py

```python
from core.env_server import create_fastapi_app
from .environment import KernelSandbox

env = KernelSandbox(gpu="RTX4090")
app = create_fastapi_app(env)
# GPU-ready sandbox! 🔥
```

✨ Define types → Implement compile/benchmark → Wrap in FastAPI → GPU Mode! ✨

🛠️ Adding Tools to Your Environment

Model Context Protocol (MCP)

⚠️ The Challenge

Problem: Agents Need Tools

Modern AI agents need access to external systems:

- Web search APIs
- File operations
- Database queries
- Git operations
- Custom integrations

But how do we expose these tools through OpenEnv's standard Action/Observation interface?

✨ The Solution

MCP = Model Context Protocol

A **standard protocol** for AI agents to discover and call tools

✅ REST-like API (JSON-RPC)
✅ Works with any AI framework
✅ Plug-and-play tool servers

Core MCP API:

```
tools/list
```

Discover available tools

```
tools/call
```

Execute a tool

OpenEnv + MCP = Universal Tool Interface for RL

Deploy with CLI

Initialize a new environment

```
openenv init my_env
cd my_env
```

Develop and Deploy to HF Spaces

```
cd my_env

# Deploy to your namespace
openenv push

# Deploy to specific repo
openenv push --repo-id username/my-env

# Deploy as private
openenv push --repo-id username/my-env --private
```

Ideas on what to build

- Environments that you want your agent to solve!
- Take inspiration from r/locallama: vendingbench, foodtricksimulater
- Realistic customer engagement (agents simulate frustrated customers)
- Push to Hub and test with Unsloth

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

- OpenEnv fundamentals, architecture
- **Local dev, Docker, and HF Spaces deployment**
- OpenEnv in Practice
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

Every HF Space provides three components:

HF Spaces are the infrastructure for OpenEnv environments

Every HF Space provides three things that OpenEnv environments need:

Component	What it provides	How to access	Used as
Server	Running environment endpoint	`https://<username>-<space-name>.hf.space`	Agent and Public API
Repository	Installable Python package	`pip install git+https://huggingface.co/spaces/<username>-<space-name>`	Code and client
Registry	Docker container image	`docker pull registry.hf.space/<username>-<space-name>:latest`	Deployment

This means a single Space deployment gives you all the components you need to use an environment in training.

1. Server: A running environment endpoint

```python
from echo_env import EchoEnv, EchoAction

# Connect directly to the running Space (WebSocket under the hood)
# Async (recommended):
async with EchoEnv(base_url="https://openenv-echo-env.hf.space") as client:
    result = await client.reset()
    result = await client.step(EchoAction(message="Hello"))

# Sync (using .sync() wrapper):
with EchoEnv(base_url="https://openenv-echo-env.hf.space").sync() as client:
    result = client.reset()
    result = client.step(EchoAction(message="Hello"))
```

Endpoints available:

Endpoint	Protocol	Description
`/ws`	**WebSocket**	Persistent session (used by client)
`/health`	HTTP GET	Health check
`/reset`	HTTP POST	Reset environment (stateless)
`/step`	HTTP POST	Execute action (stateless)
`/state`	HTTP GET	Get current state
`/docs`	HTTP GET	OpenAPI documentation
`/web`	HTTP GET	Interactive web UI

Example: Checking if the space is running

```
curl https://openenv-echo-env.hf.space/health
# {"status": "healthy"}
```

2. Repository: Installable Python package

Every Space is a Git repository. OpenEnv environments include a `pyproject.toml`, making them pip-installable directly from the Space URL.

```
# Install client package from Space
pip install git+https://huggingface.co/spaces/openenv/echo-env
```

This installs:

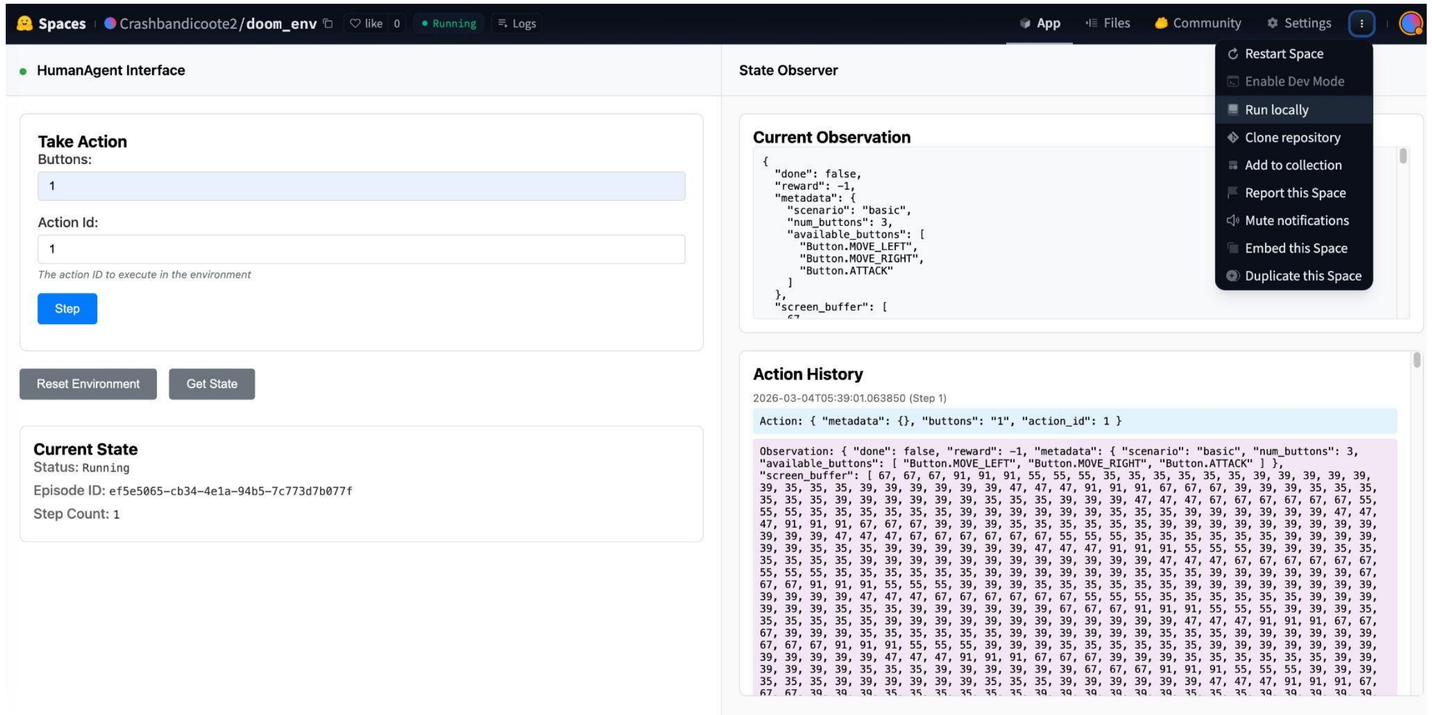
- **Client class** (`EchoEnv`) — Handles HTTP/WebSocket communication
- **Models** (`EchoAction`, `EchoObservation`) — Typed action and observation classes
- **Utilities** — Any helper functions the environment provides

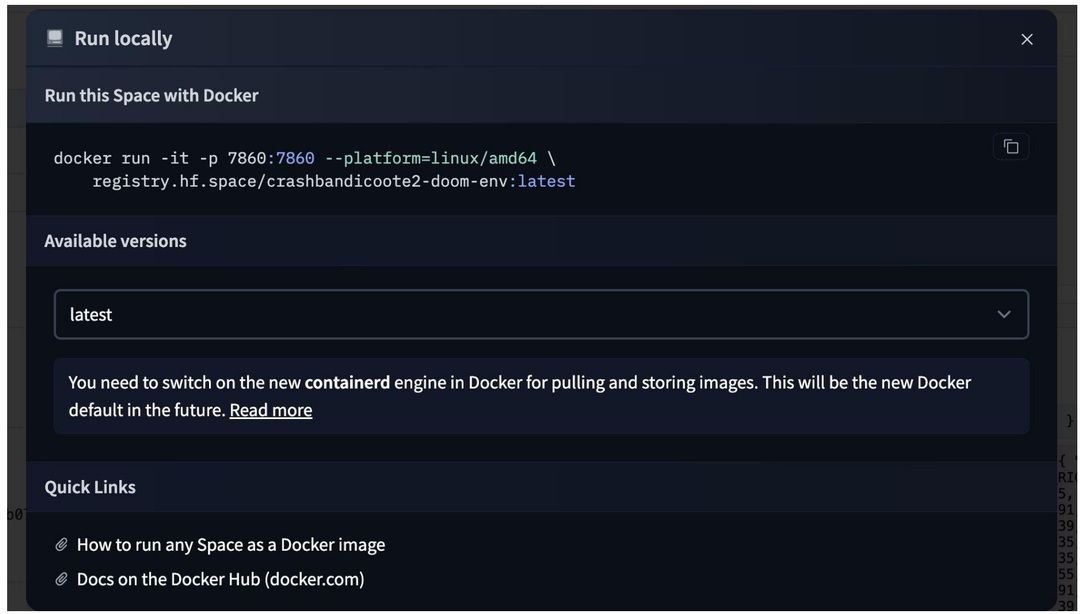
After installation:

```
from envs.echo_env import EchoEnv, EchoAction, EchoObservation

# Now you have typed classes for the environment
action = EchoAction(message="Hello")
```

3. Registry: Docker container image

```
# Pull the image
docker pull registry.hf.space/openenv-echo-env:latest

# Run locally on port 8001
docker run -d -p 8001:8000 registry.hf.space/openenv-echo-env:latest
```





Typical workflow:

```python
import asyncio
from echo_env import EchoEnv, EchoAction

async def main():
    # Development: connect to remote Space
    async with EchoEnv(base_url="https://openenv-echo-env.hf.space") as client:
        result = await client.reset()

    # Production: run locally for speed
    # docker run -d -p 8001:8000 registry.hf.space/openenv-echo-env:latest
    async with EchoEnv(base_url="http://localhost:8001") as client:
        result = await client.reset()

    # Or let the client manage Docker for you
    client = await EchoEnv.from_env("openenv/echo-env")  # Auto-pulls and runs
    async with client:
        result = await client.reset()

asyncio.run(main())

# For sync usage, use the .sync() wrapper:
with EchoEnv(base_url="http://localhost:8001").sync() as client:
    result = client.reset()
```

Local Development

- **Clone and run the environment locally**

```
# Clone from HF Space
git clone https://huggingface.co/spaces/burtenshaw/openenv-benchmark
cd openenv-benchmark

# Install in editable mode
uv sync

# Start server
uv run server

# Run isolated from remote Space
uv run --isolated --project https://huggingface.co/spaces/burtenshaw/openenv-benchmark server
```

Local Development

- Uvicorn directly in python

```
# Full control over uvicorn options
uvicorn benchmark.server.app:app --host "$HOST" --port "$PORT" --workers "$WORKERS"

# With reload for development
uvicorn benchmark.server.app:app --host 0.0.0.0 --port 8000 --reload

# Multi-Worker Mode For better concurrency:
uvicorn benchmark.server.app:app --host 0.0.0.0 --port 8000 --workers 4
```

Local Development

- **Run the environment locally from the space**

```
# Run the environment locally from the space
docker run -d -p 8000:8000 registry.hf.space/openenv-echo-env:latest
```

Build Image

```
# Clone from HF Space
git clone https://huggingface.co/spaces/burtenshaw/openenv-benchmark
cd openenv-benchmark

# Using OpenEnv CLI (recommended)
openenv build -t openenv-benchmark:latest

# Or with Docker directly
docker build -t openenv-benchmark:latest -f server/Dockerfile .
```

Run Container

```
# Basic run
docker run -d -p 8000:8000 my-env:latest

# With environment variables
docker run -d -p 8000:8000 \
    -e WORKERS=4 \
    -e MAX_CONCURRENT_ENVS=100 \
    my-env:latest

# Named container for easy management
docker run -d --name my-env -p 8000:8000 my-env:latest
```

Deploy with CLI

Initialize a new environment

```
openenv init my_env
cd my_env
```

Develop and Deploy to HF Spaces

```
cd my_env

# Deploy to your namespace
openenv push

# Deploy to specific repo
openenv push --repo-id username/my-env

# Deploy as private
openenv push --repo-id username/my-env --private
```

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- **OpenEnv in Practice**
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

OpenEnv in Practice

```
Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
❯ uv venv
Using CPython 3.11.14
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate

Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)

Downloads/Projects/SF_Hackathon_OpenEnv via 🐍 v3.11.14 (SF_Hackathon_OpenEnv) on ☁ (us-east-1)
❯ uv pip list

Downloads/Projects/SF_Hackathon_OpenEnv via 🐍 v3.11.14 (SF_Hackathon_OpenEnv) on ☁ (us-east-1)
❯ uv pip install openenv-core
Resolved 86 packages in 309ms
Installed 86 packages in 262ms
```

Use uv venv

Or

```
Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
❯ conda create -n openenv_hackathon python=3.12
```

```
Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
❯ conda activate openenv_hackathon
(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via Ⓒ openenv_hackathon on ☁ (us-east-1)
❯
```

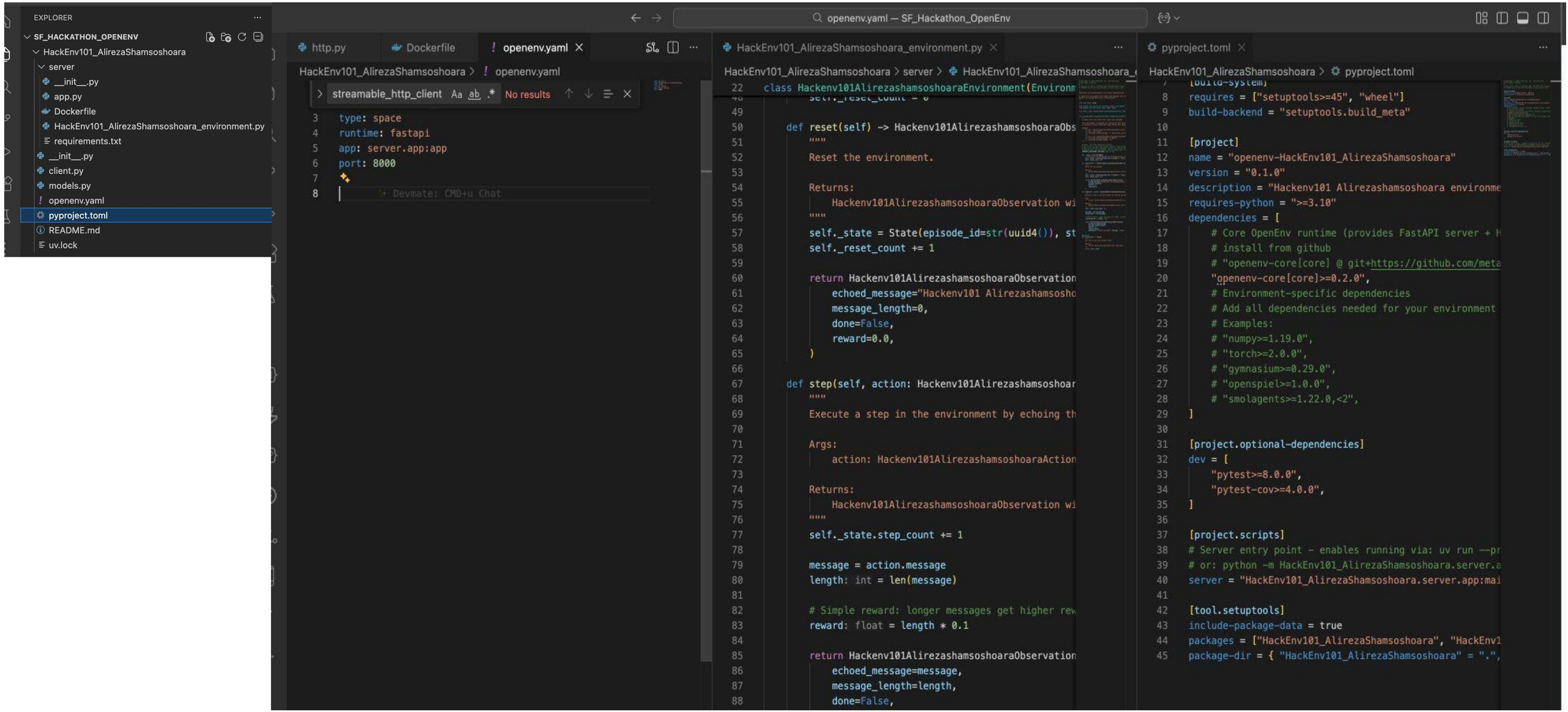
Use conda env

```
(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via Ⓒ openenv_hackathon on ☁ (us-east-1)
❯ uv pip install openenv-core
Using Python 3.12.12 environment at: /Users/alisol/miniconda3/envs/openenv_hackathon
Resolved 85 packages in 833ms
Prepared 30 packages in 4.37s
Installed 84 packages in 453ms
```

Initialize an Environment

```
Downloads/Projects/SF_Hackathon_OpenEnv via © openenv_hackathon on ☁ (us-east-1)
❯ openenv init HackEnv101_AlirezaShamsoshoara
Creating OpenEnv environment 'HackEnv101_AlirezaShamsoshoara'...
✓ Created 11 files

Generating uv.lock...
✓ Generated uv.lock

Environment created successfully at:
/Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara

Next steps:
  cd /Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara
  # Edit your environment implementation in server/HackEnv101_AlirezaShamsoshoara_environment.py
  # Edit your models in models.py
  # Install dependencies: uv sync

  # To integrate into OpenEnv repo:
  # 1. Copy this directory to <repo_root>/envs/HackEnv101_AlirezaShamsoshoara_env
  # 2. Build from repo root: docker build -t HackEnv101_AlirezaShamsoshoara_env:latest -f
envs/HackEnv101_AlirezaShamsoshoara_env/server/Dockerfile .
  # 3. Run your image: docker run -p 8000:8000 HackEnv101_AlirezaShamsoshoara_env:latest
(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via © openenv_hackathon on ☁ (us-east-1) took 2s
```

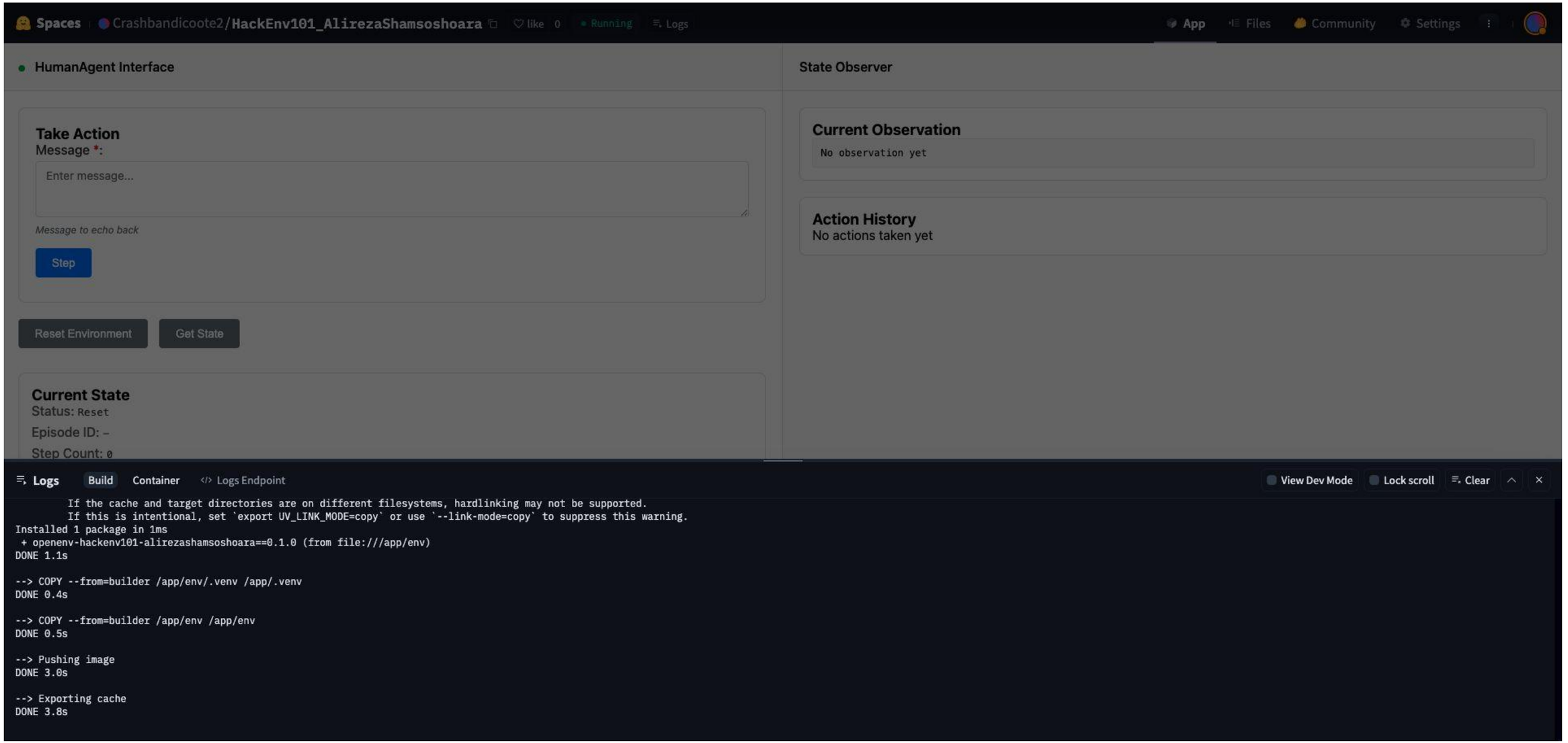
Initialize an Environment



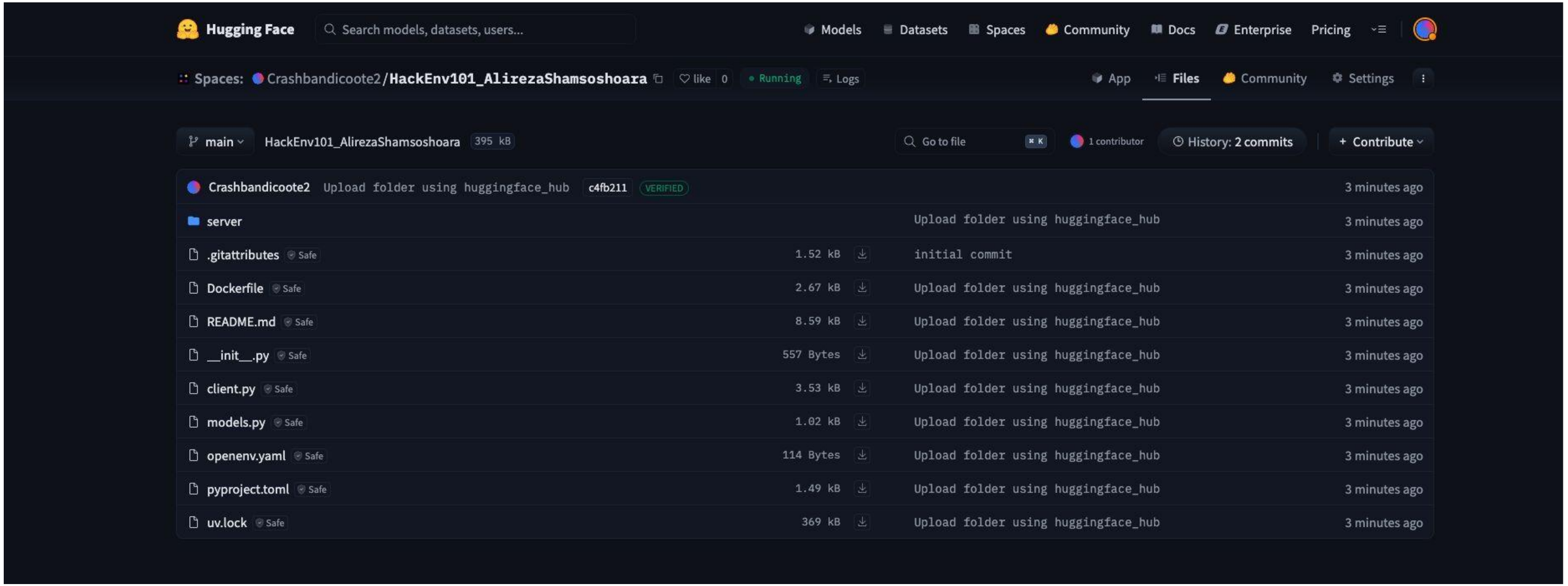
Develop and Push your created environment

```
Downloads/Projects/SF_Hackathon_OpenEnv via Ⓒ openenv_hackathon on ☁️ (us-east-1)
❯ cd HackEnv101_AlirezaShamsoshoara
(openenv_hackathon)
Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara is 📦 v0.1.0 via 🐍 v3.12.12 via Ⓒ openenv_hackathon on ☁️ (us-east-1)
❯ openenv push
Validating OpenEnv environment in /Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara...
⚠ Recommended directory missing: outputs/
✓ Found OpenEnv environment: HackEnv101_AlirezaShamsoshoara
✓ Authenticated as: Crashbandicoote2
Preparing files for Hugging Face deployment (with web interface)...
Moved Dockerfile to repository root for deployment
✓ Updated Dockerfile: enabled web interface
Creating/verifying space: Crashbandicoote2/HackEnv101_AlirezaShamsoshoara
✓ Space Crashbandicoote2/HackEnv101_AlirezaShamsoshoara is ready
Uploading files to Crashbandicoote2/HackEnv101_AlirezaShamsoshoara...
✓ Upload completed successfully
Space URL: https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsoshoara

✓ Deployment complete!
Visit your space at: https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsoshoara
(openenv_hackathon)
Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara is 📦 v0.1.0 via 🐍 v3.12.12 via Ⓒ openenv_hackathon on ☁️ (us-east-1) took 6s
❯
```

Your Env on HF Spaces



Your Env on HF Spaces



Run Locally or Clone the Environment

Run locally

Run this Space with Docker

```
docker run -it -p 7860:7860 --platform=linux/amd64 \
    registry.hf.space/crashbandicoote2-hackenv101-alirezashamsoshoara:latest
```

Clone this Space repository

HTTPS SSH

```
# Make sure git-xet is installed (https://hf.co/docs/hub/git-xet)
brew install git-xet
git xet install
```

```
git clone https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsoshoara
```

```
# If you want to clone without large files - just their pointers
GIT_LFS_SKIP_SMUDGE=1 git clone https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsos
```

cURL Homebrew uv uvx

```
# Make sure the hf CLI is installed
curl -LsSf https://hf.co/cli/install.sh | bash
```

```
# Download the Space
hf download Crashbandicoote2/HackEnv101_AlirezaShamsoshoara --repo-type=space
```

ECOSYSTEM

Available Environments

100s of environments on HF Hub with a unified API — from game theory to autonomous driving

🎲
OpenSpiel
70+ game theory games: chess, poker, Go, tic-tac-toe
`game-theory`

💻
Coding Env
Python code execution with smolagents sandbox
`code-exec`

🌐
BrowserGym
Browser automation — navigate, click, fill forms
`web`

🚀
Atari
Classic Atari via Gymnasium — Breakout, Pong
`arcade`

🔎
WebSearch
Web search, browse results, extract info
`search`

🚗
CARLA
Autonomous driving with ethical AI benchmarks
`featured`

📈
FinQA
Financial QA benchmarking environment
`featured`

📈
FinRL
Stock trading, portfolio management
`finance`

⚔
Chess Env
Chess with LLMJudge rubric integration
`featured`

💬
TextArena
Multi-agent text-based competitive games
`featured`

🛠
Git Env
Git operations — commits, branches, merges
`devtools`

🚦
SUMO RL
Traffic signal control simulation
`traffic`

📅
Calendar Env
Calendar scheduling with LLMClient
`featured`

🤖
DM Control
DeepMind Control Suite robotics
`featured`

🔨
REPL Env
Interactive REPL with recursion support
`featured`

😇
Maze Env
Navigate grid mazes to the exit cell
`featured`

🧠
Reasoning Gym
Logic puzzles, math, reasoning challenges
`reasoning`

💬
Chat Env
Conversational dialogue training
`conversation`

🐍
Snake / Connect4
Classic games for testing RL pipelines
`classic`

+
+ 100s More on HF Hub
Julia, Unity, OpenApp, Grid World, KernRL, Wildfire, TBench2, DIPG Safety, Echo, and community contributions...
`growing`

100s	1	6+	30
ON HF HUB	UNIFIED API	RL FRAMEWORKS	IN REPO

Same 4 methods everywhere: reset() · step() · state() · close()
Build once, train with any framework, deploy anywhere.

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- **Training (TRL & Unsloth)**
- How-to-Access-Infrastructure

- **Training examples:** https://github.com/meta-pytorch/OpenEnv/tree/main/tutorial/examples
- **Unsloth example (2048):** https://github.com/meta-pytorch/OpenEnv/blob/main/tutorial/examples/unsloth_2048.ipynb
- **Wordle example-trl:** https://github.com/meta-pytorch/OpenEnv/blob/main/tutorial/examples/wordle.py
- **trl:**
  - ➢ **TRL OpenEnv Doc:** https://huggingface.co/docs/trl/en/openenv
  - ➢ **Sudoku example**: https://github.com/huggingface/trl/blob/main/examples/notebooks/openenv_sudoku_grpo.ipynb
  - ➢ **Wordle example**: https://github.com/huggingface/trl/blob/main/examples/notebooks/openenv_wordle_grpo.ipynb
  - ➢ **More TRL examples:** https://github.com/huggingface/trl/tree/main/examples/scripts/openenv

TRAINING INTEGRATION

Training with TRL (GRPO)

HuggingFace TRL integrates natively with OpenEnv environments for GRPO training

trl_grpo_train.py — TRL + OpenEnv

```python
from trl import GRPOTrainer, GRPOConfig
from transformers import AutoModelForCausalLM
from hack_env import HackEnv

# 1. Load model
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B-Instruct"
)

# 2. Define reward function using OpenEnv
def openenv_reward(completions, **kwargs):
    rewards = []
    for completion in completions:
        with HackEnv(base_url="...").sync() as env:
            env.reset()
            result = env.step(completion)
            rewards.append(result.reward)
    return rewards

# 3. Configure GRPO training
config = GRPOConfig(
    output_dir="./hack-agent",
    num_generations=4,
    max_completion_length=512,
    per_device_train_batch_size=2,
    learning_rate=5e-6,
)

# 4. Train!
trainer = GRPOTrainer(
    model=model,
    reward_funcs=openenv_reward,
    args=config,
    train_dataset=dataset,
)
trainer.train()
```

GRPO TRAINING FLOW

What is GRPO?

Group Relative Policy Optimization — generates N completions per prompt, ranks them by reward, and updates the policy to prefer higher-reward completions. No critic network needed.

Why OpenEnv + TRL?

OpenEnv provides the `reward_funcs` callback. Each completion is evaluated in an isolated environment. Rewards are computed by the environment's rubric system, not by external heuristics.

Batch Training Pattern

For parallel evaluation, spin up N environment instances. Use `asyncio.gather` or `EnvPool` to evaluate all completions concurrently.

Official Docs

Full TRL + OpenEnv integration guide:
`huggingface.co/docs/trl/openenv`

TRAINING INTEGRATION

Training with Unsloth

Fast RL fine-tuning with Unsloth's optimized training + OpenEnv environments

unsloth_train.py — Unsloth + OpenEnv

```python
from unsloth import FastLanguageModel
from trl import GRPOTrainer, GRPOConfig
from hack_env import HackEnv

# 1. Load model with Unsloth (2x faster)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Qwen2.5-7B-Instruct",
    max_seq_length=2048,
    load_in_4bit=True,      # 4-bit quantization
)

# 2. Apply LoRA adapters
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj",
                    "v_proj", "o_proj"],
)

# 3. OpenEnv reward function
def env_reward(completions, **kw):
    rewards = []
    for c in completions:
        with HackEnv("http://localhost:8000"
                     ).sync() as env:
            env.reset()
            r = env.step(c)
            rewards.append(r.reward)
    return rewards

# 4. Train with GRPO
trainer = GRPOTrainer(
    model=model,
    reward_funcs=env_reward,
    args=GRPOConfig(
        learning_rate=5e-6,
        num_generations=4,
```

2x FASTER TRAINING

70% LESS MEMORY

4-bit QUANTIZATION

LoRA EFFICIENT TUNING

Why Unsloth?

Unsloth provides **2x faster** training and **70% less memory** through custom CUDA kernels. Works as a drop-in replacement — same TRL API, just faster.

The Pattern

1. Load model via `FastLanguageModel` (with 4-bit quantization)
2. Apply LoRA adapters for parameter-efficient training
3. Use OpenEnv as the reward function
4. Train with standard `GRPOTrainer`

Google Colab Ready

Run on a free T4 GPU. Unsloth + OpenEnv Colab notebook available for the 2048 game environment with 20B parameter models.

Also Compatible With

TRL · torchforge · SkyRL · ART · Oumi · veRL

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- Training (TRL & Unsloth)
- **How-to-Access-Infrastructure (Training & Inference)**

Use HF infra to run your training

- Hugging Face Jobs provide compute for AI and data workflows
- You can run jobs using the hf CLI, the huggingface_hub Python client, or the Jobs HTTP API
- Jobs support any hardware from CPUs to H100s & TPUs, with pay-as-you-go

References:

Hugging Face billing and credit	https://huggingface.co/settings/billing
Hugging Face jobs	https://huggingface.co/settings/jobs
Hugging Face jobs docs	https://huggingface.co/docs/hub/jobs
Job CLI doc	https://huggingface.co/docs/huggingface_hub/guides/cli#hf-jobs https://huggingface.co/docs/huggingface_hub/guides/jobs
Run and manage	https://huggingface.co/docs/hub/jobs-pricing
Jobs Pricing and	https://huggingface.co/docs/hub/jobs-examples
Billing Jobs examples	

- Depends on your model size, you can choose your GPU model

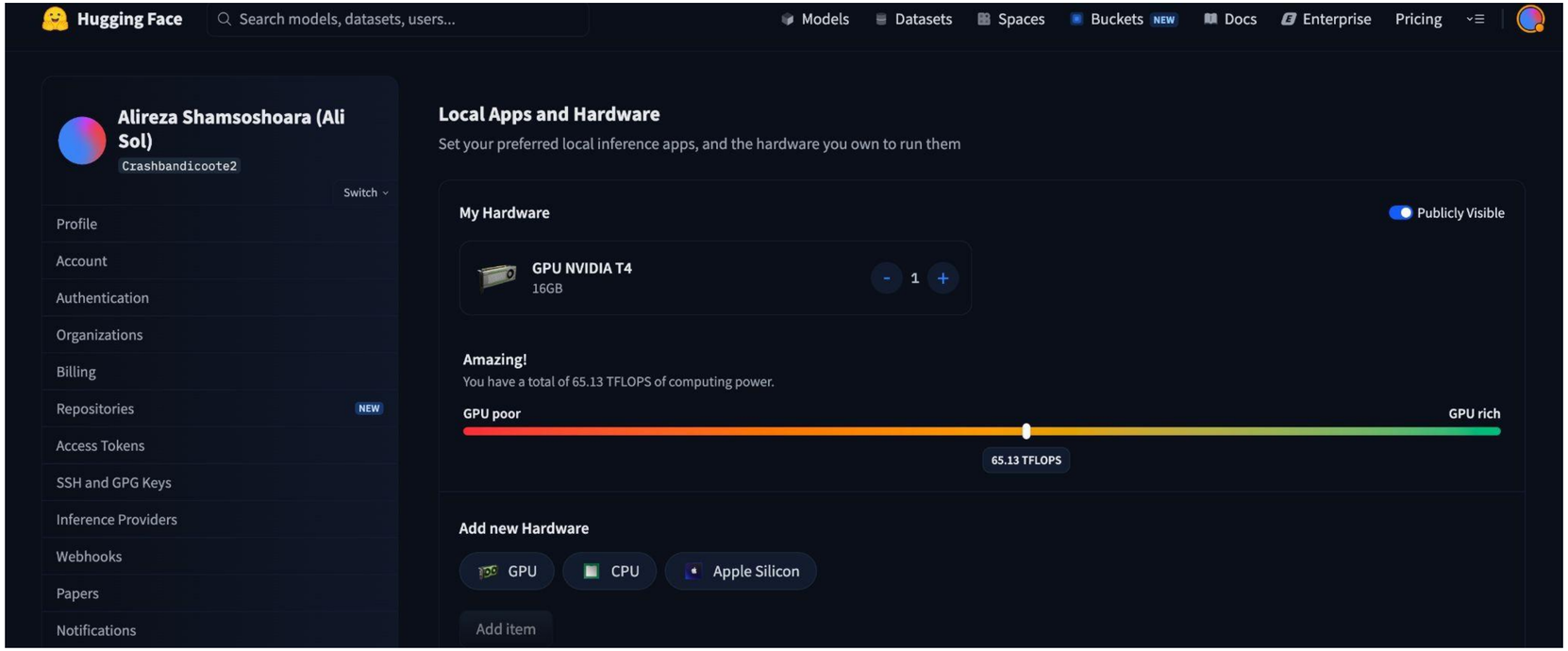
- We suggest choose your GPU wisely so you can run your training / inference with your partner for a reasonable time (credit)

- A T4 GPU (small / medium) would be a good choice

Hardware	CPU	Memory	Hourly Price
CPU Basic	2 vCPU	16 GB	$0.01
CPU Upgrade	8 vCPU	32 GB	$0.03
CPU XL	16 vCPU	124 GB	$1.00
CPU Performance	32 vCPU	256 GB	$1.90

GPU

Hardware	CPU	Memory	GPU Memory	Hourly Price
Nvidia T4 - small	4 vCPU	15 GB	16 GB	$0.40
Nvidia T4 - medium	8 vCPU	30 GB	16 GB	$0.60
1x Nvidia L4	8 vCPU	30 GB	24 GB	$0.80
4x Nvidia L4	48 vCPU	186 GB	96 GB	$3.80
1x Nvidia L40S	8 vCPU	62 GB	48 GB	$1.80
4x Nvidia L40S	48 vCPU	382 GB	192 GB	$8.30

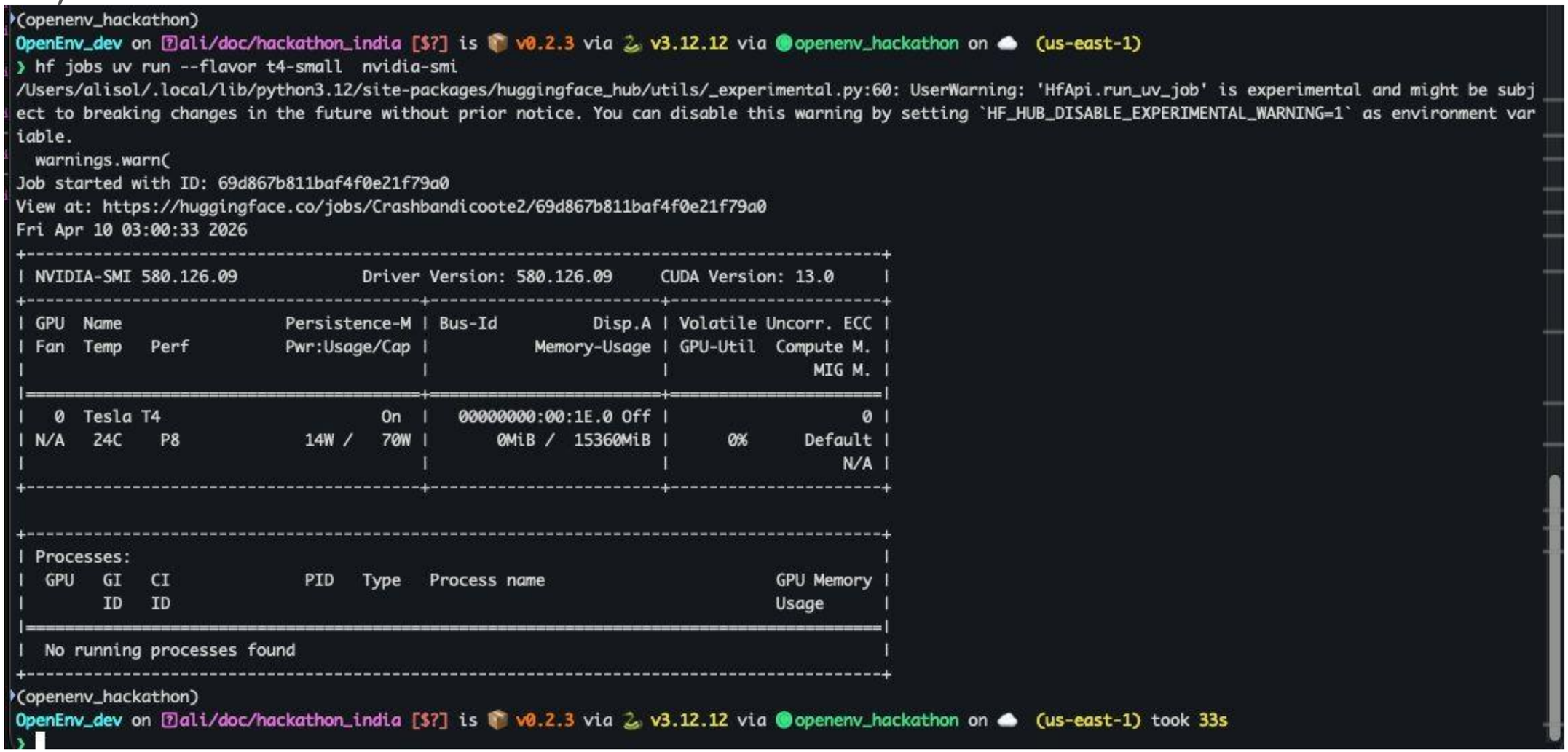


- You can see what GPUs you can request (`hf jobs hardware`):

```
(openenv_hackathon)
OpenEnv_dev on ali/doc/hackathon_india [$?] is v0.2.3 via v3.12.12 via openenv_hackathon on (us-east-1)
❯ hf jobs hardware
NAME             PRETTY NAME                 CPU       RAM      ACCELERATOR          COST/MIN COST/HOUR
---------------- --------------------------- --------- -------- -------------------- -------- ---------
cpu-basic        CPU Basic                   2 vCPU    16 GB    N/A                  $0.0002  $0.01
cpu-upgrade      CPU Upgrade                 8 vCPU    32 GB    N/A                  $0.0005  $0.03
cpu-performance  CPU Performance             32 vCPU   256 GB   N/A                  $0.3117  $18.70
cpu-xl           CPU XL                      16 vCPU   124 GB   N/A                  $0.0167  $1.00
t4-small         Nvidia T4 - small           4 vCPU    15 GB    1x T4 (16 GB)        $0.0067  $0.40
t4-medium        Nvidia T4 - medium          8 vCPU    30 GB    1x T4 (16 GB)        $0.0100  $0.60
a10g-small       Nvidia A10G - small         4 vCPU    15 GB    1x A10G (24 GB)      $0.0167  $1.00
a10g-large       Nvidia A10G - large         12 vCPU   46 GB    1x A10G (24 GB)      $0.0250  $1.50
a10g-largex2     2x Nvidia A10G - large      24 vCPU   92 GB    2x A10G (48 GB)      $0.0500  $3.00
a10g-largex4     4x Nvidia A10G - large      48 vCPU   184 GB   4x A10G (96 GB)      $0.0833  $5.00
a100-large       Nvidia A100 - large         12 vCPU   142 GB   1x A100 (80 GB)      $0.0417  $2.50
a100x4           4x Nvidia A100              48 vCPU   568 GB   4x A100 (320 GB)     $0.1667  $10.00
a100x8           8x Nvidia A100              96 vCPU   1136 GB  8x A100 (640 GB)     $0.3333  $20.00
h200             Nvidia H200                 23 vCPU   256 GB   1x H200 (141 GB)     $0.0833  $5.00
h200x2           Nvidia H200                 46 vCPU   512 GB   2x H200 (282 GB)     $0.1667  $10.00
h200x4           Nvidia H200                 92 vCPU   1024 GB  4x H200 (564 GB)     $0.3333  $20.00
h200x8           Nvidia H200                 184 vCPU  2048 GB  8x H200 (1128 GB)    $0.6667  $40.00
l4x1             1x Nvidia L4                8 vCPU    30 GB    1x L4 (24 GB)        $0.0133  $0.80
l4x4             4x Nvidia L4                48 vCPU   186 GB   4x L4 (96 GB)        $0.0633  $3.80
l40sx1           1x Nvidia L40S              8 vCPU    62 GB    1x L40S (48 GB)      $0.0300  $1.80
l40sx4           4x Nvidia L40S              48 vCPU   382 GB   4x L40S (192 GB)     $0.1383  $8.30
l40sx8           8x Nvidia L40S              192 vCPU  1534 GB  8x L40S (384 GB)     $0.3917  $23.50
(openenv_hackathon)
OpenEnv_dev on ali/doc/hackathon_india [$?] is v0.2.3 via v3.12.12 via openenv_hackathon on (us-east-1)
❯
```

-

```
    --model_name_or_path Qwen/Qwen2-0.5B ...
    )
```



Still have Questions?

Please mention it in discord
(India-OpenEnv-Hackathon Channels)
and we do our best to answer